C0C25077 / ProjExD_Group16

Code Pull requests Actions Projects Wiki Security and quality More

wallet ProjExD_Group16 / README.md Go to file T

C0C25077 README追記 a33669c · now

Preview Code Blame 46 lines (36 loc) · 2.67 KB Raw

ProjExD_Group16

#〇〇〇 大戦争

実行環境の必要条件

- python >= 3.10
- pygame >= 2.1

ゲームの概要

- 右側の自陣からキャラクターを出撃させ、左側の敵陣を攻略するタワーディフェンスゲーム
- 参考URL：サイトタイトル

ゲームの遊び方

- （操作方法など）

ゲームの実装

共通基本機能

- 背景画像と主人公キャラクターの描画

分担追加機能

- 機能名（担当：名前）：説明 ・追加機能１（もろ）：お金上昇＆財布レベルアップ ➡キーかボタンで所持金の上限と増加速度を強化。強化費用が上がり、最大レベルを設定する。UIにレベル表示 ・追加機能１（もろ）：お金上昇＆財布レベルアップ ➡ キー入力やUIボタンクリックにより、お金の管理パラメータをリアルタイムで強化するシステム。 **操作**: `W` キー / `0` キー / 画面右下「財布LV UP」ボタン **仕様**: `Wallet` クラスを拡張し、財布レベル（`level`）、最大レベル（`max_level`）、強化費用計算（`get_level_up_cost()`）、レベルアップ処理（`level_up()`）を実装。 レベルアップごとに「所持金上限（`+350`）」と「毎フレームの収入量（`+0.35`）」が増加。

レベルに応じたコスト（レベル×100円）を設定し、所持金が足りない場合や最大レベル到達時はボタンがグレーアウトして発動を制限。 画面右上のお金表示に現在のレベルを追記し、ボタン内に操作キーと必要コストを描画。

・追加機能２（かわひら）：キャラクター全般 ➡範囲攻撃、遠距離攻撃、味方の足を止める敵、体力が大きいボス敵など。Unit を継承したクラスで作る

・追加機能３（りつき）：ステージ・ウェーブ管理 ➡ステージ選択、敵の出現パターン（ウェーブ制）、一定時間後にボス登場、ステージごとに敵城のHPを変える

・追加機能４（やくしじ）：画像、アニメ、BGM・効果音 ➡四角を画像に差し替え、歩行アニメ、攻撃時の効果音、BGM、ミュートキー。素材は自作かフリーのもの

追加機能５（こざわ）：お金稼ぎミニゲーム ➡画面上から落ちてくるコインをクリックでキャッチすると、お金が増えます。収入を待つだけでなく、操作で稼ぐ手応えが出ます。実装は Coin クラスのスプライトと衝突判定です。

メモ

- （変数名や設計のルールなど、メンバーへの連絡）

```python
"""
にゃんこ大戦争風ゲーム

右側の自陣から味方ユニット（ネコ）を出撃させ，左側の敵陣を攻略するゲーム．
お金は時間とともに増え，お金とクールダウンの許す範囲でユニットを出撃できる．
ユニットは敵に当たると立ち止まって攻撃し，敵陣のHPを0にすれば勝利．

操作：
- 数字キー 1〜3 または画面下ボタン：ユニット出撃
- Wキー / 0キー または画面右下ボタン：財布レベルアップ（お金の増加速度と上限アップ）
"""

import os
import random
import sys
import time
from typing import List, Optional

import pygame as pg

# 定数定義
WIDTH, HEIGHT = 1100, 650
GROUND_Y = 500  # 地面の高さ（ユニットの足元のy座標）
FPS = 50
ENEMY_INTERVAL = 150  # 敵が出現する間隔（フレーム数）
BASE_HP = 1000

os.chdir(os.path.dirname(os.path.abspath(__file__)))

# 味方ユニットの性能（追加機能として種類を増やしやすいように辞書で管理）
CAT_SPECS = [
    {
        "name": "ネコ",
        "cost": 50,
        "hp": 100,
        "atk": 10,
        "speed": 2.0,
        "range": 20,
        "interval": 40,
        "cooldown": 100,
        "color": (255, 220, 160),
        "size": (40, 50),
    },
    {
```

```
            "name": "タンク",
            "cost": 100,
            "hp": 500,
            "atk": 6,
            "speed": 1.2,
            "range": 20,
            "interval": 50,
            "cooldown": 200,
            "color": (200, 200, 255),
            "size": (50, 70),
        },
        {
            "name": "キック",
            "cost": 150,
            "hp": 120,
            "atk": 35,
            "speed": 1.8,
            "range": 60,
            "interval": 70,
            "cooldown": 250,
            "color": (255, 150, 150),
            "size": (40, 60),
        },
    ]

    # 敵ユニットの性能
    ENEMY_SPECS = [
        {
            "name": "ワンコ",
            "hp": 120,
            "atk": 8,
            "speed": 1.5,
            "range": 20,
            "interval": 45,
            "color": (240, 240, 240),
            "size": (40, 40),
        },
        {
            "name": "ブタ",
            "hp": 500,
            "atk": 20,
            "speed": 1.0,
            "range": 20,
            "interval": 60,
            "color": (255, 170, 200),
            "size": (60, 80),
        },
    ]


    def get_font(size: int) -> pg.font.Font:
        """日本語を表示できるフォントを探して返す.

        Args:
            size (int): フォントサイズ

        Returns:
            pg.font.Font: Fontオブジェクト（見つからない場合は標準フォント）
```

```
        """
        name = pg.font.match_font(
            "meiryo,yugothic,msgothic,notosanscjkjp,hiraginosans,takaopgothic"
        )
        return pg.font.Font(name, size)


def draw_hp_bar(screen: pg.Surface, rct: pg.Rect, hp: int, max_hp: int) -> None:
    """対象の頭上にHPバーを描画する.

    Args:
        screen (pg.Surface): 画面Surface
        rct (pg.Rect): 対象のRect
        hp (int): 現在のHP
        max_hp (int): 最大HP
    """
    ratio = max(0, hp) / max_hp
    bar = pg.Rect(rct.left, rct.top - 10, rct.width, 6)
    pg.draw.rect(screen, (80, 80, 80), bar)
    pg.draw.rect(
        screen,
        (0, 200, 0),
        (bar.left, bar.top, int(bar.width * ratio), bar.height),
    )


class Base:
    """自陣・敵陣（城）に関するクラス."""

    def __init__(self, is_cat: bool) -> None:
        """引数 is_cat：Trueなら自陣（右側），Falseなら敵陣（左側）."""
        self.hp = BASE_HP
        self.max_hp = BASE_HP
        self.image = pg.Surface((80, 200))
        self.image.fill((100, 150, 255) if is_cat else (255, 120, 120))
        pg.draw.rect(self.image, (0, 0, 0), self.image.get_rect(), 3)
        self.rect = self.image.get_rect()
        self.rect.bottom = GROUND_Y
        self.rect.centerx = WIDTH - 80 if is_cat else 80

    def draw(self, screen: pg.Surface) -> None:
        """城とHPバーを描画する.

        Args:
            screen (pg.Surface): 画面Surface
        """
        screen.blit(self.image, self.rect)
        draw_hp_bar(screen, self.rect, self.hp, self.max_hp)


class Unit(pg.sprite.Sprite):
    """味方・敵のユニットに関するクラス."""

    def __init__(self, spec: dict, is_cat: bool, x: int) -> None:
        """ユニット初期化.

        Args:
            spec (dict): ユニットの性能を表す辞書
```

```
161             is_cat (bool): Trueなら味方（左へ進む），Falseなら敵（右へ進む）
162             x (int): 出現位置のx座標
163         """
164         super().__init__()
165         self.spec = spec
166         self.hp = spec["hp"]
167         self.dir = -1 if is_cat else +1
168         self.atk_timer = spec["interval"]  # 最初の1発はすぐ撃てるようにする
169         self.image = pg.Surface(spec["size"])
170         self.image.fill(spec["color"])
171         pg.draw.rect(self.image, (0, 0, 0), self.image.get_rect(), 2)
172         self.rect = self.image.get_rect()
173         self.rect.bottom = GROUND_Y
174         if is_cat:
175             self.rect.right = x
176         else:
177             self.rect.left = x
178         self.x = float(self.rect.centerx)  # 小数の速度を扱うため，x座標は別に保持する
179 
180     def can_reach(self, rct: pg.Rect) -> bool:
181         """対象のRectが攻撃射程内にあるかを判定する．
182 
183         Args:
184             rct (pg.Rect): 敵ユニットまたは敵陣のRect
185 
186         Returns:
187             bool: 射程内ならTrue
188         """
189         if self.dir < 0:
190             return rct.right >= self.rect.left - self.spec["range"]
191         return rct.left <= self.rect.right + self.spec["range"]
192 
193     def find_target(
194         self, opponents: pg.sprite.Group, base: Base
195     ) -> Optional["Unit | Base"]:
196         """攻撃対象を探す（射程内のユニットのうち最も近いもの，いなければ敵城）．
197 
198         Args:
199             opponents (pg.sprite.Group): 相手側ユニットのグループ
200             base (Base): 相手側の城
201 
202         Returns:
203             Optional[Unit | Base]: 攻撃対象（いなければNone）
204         """
205         cands = [o for o in opponents if self.can_reach(o.rect)]
206         if cands:
207             return min(cands, key=lambda o: abs(o.rect.centerx - self.rect.centerx))
208         if self.can_reach(base.rect):
209             return base
210         return None
211 
212     def update(self, opponents: pg.sprite.Group, base: Base) -> None:
213         """HP判定，攻撃，移動を行う．
214 
215         Args:
216             opponents (pg.sprite.Group): 相手側ユニットのグループ
217             base (Base): 相手側の城
218         """
```

```
            if self.hp <= 0:
                self.kill()
                return
            target = self.find_target(opponents, base)
            if target is None:
                self.x += self.dir * self.spec["speed"]
                self.rect.centerx = int(self.x)
                return
            self.atk_timer += 1
            if self.atk_timer >= self.spec["interval"]:
                target.hp -= self.spec["atk"]
                self.atk_timer = 0


class Wallet:
    """お金（所持金）および財布のレベルアップに関するクラス."""

    def __init__(self) -> None:
        """初期パラメータの設定."""
        self.money = 100.0
        self.level = 1
        self.max_level = 8
        self.max_money = 500  # 初期上限
        self.income = 0.4  # 1フレームあたりの初期増加量

    def update(self) -> None:
        """お金を時間経過で増やす（上限あり）."""
        self.money = min(float(self.max_money), self.money + self.income)

    def pay(self, cost: int) -> bool:
        """所持金が足りれば支払う.

        Args:
            cost (int): 支払う金額

        Returns:
            bool: 支払えたらTrue，足りなければFalse
        """
        if self.money < cost:
            return False
        self.money -= cost
        return True

    def get_level_up_cost(self) -> int:
        """現在のレベルに応じたレベルアップに必要な費用を取得する.

        Returns:
            int: レベルアップに必要な費用（最大レベルの場合は0）
        """
        if self.level >= self.max_level:
            return 0
        return 100 * self.level

    def level_up(self) -> bool:
        """所持金が足りれば財布レベルを上げ，所持金上限と増加速度を強化する.

        Returns:
            bool: レベルアップに成功したらTrue，失敗・レベルMAXならFalse
        """
```

```
            if self.level >= self.max_level:
                return False
            cost = self.get_level_up_cost()
            if not self.pay(cost):
                return False

            self.level += 1
            self.max_money += 350  # 所持金上限の増加
            self.income += 0.35   # 毎フレームの収入増加量アップ
            return True


    def make_buttons() -> List[pg.Rect]:
        """出撃ボタンのRectのリストを作る．

        Returns:
            List[pg.Rect]: 味方ユニットの種類ごとのボタンRect
        """
        return [pg.Rect(20 + i * 170, 540, 160, 90) for i in range(len(CAT_SPECS))]


    def make_wallet_button() -> pg.Rect:
        """財布レベルアップボタンのRectを作る．

        Returns:
            pg.Rect: 財布レベルアップボタンのRect
        """
        return pg.Rect(WIDTH - 180, 540, 160, 90)


    def try_spawn(
        idx: int,
        wallet: Wallet,
        cooldowns: List[int],
        cats: pg.sprite.Group,
        cat_base: Base,
    ) -> None:
        """お金とクールダウンの条件を満たせば，味方ユニットを出撃させる．

        Args:
            idx (int): CAT_SPECSの番号
            wallet (Wallet): 所持金オブジェクト
            cooldowns (List[int]): ユニットごとの残りクールダウン
            cats (pg.sprite.Group): 味方ユニットのグループ
            cat_base (Base): 自陣
        """
        spec = CAT_SPECS[idx]
        if cooldowns[idx] > 0 or not wallet.pay(spec["cost"]):
            return
        cats.add(Unit(spec, True, cat_base.rect.left))
        cooldowns[idx] = spec["cooldown"]


    def draw_ui(
        screen: pg.Surface,
        font: pg.font.Font,
        wallet: Wallet,
        cooldowns: List[int],
```

```python
        buttons: List[pg.Rect],
        wallet_btn: pg.Rect,
    ) -> None:
        """所持金表示・出撃ボタン・財布レベルアップボタンを描画する.

        Args:
            screen (pg.Surface): 画面Surface
            font (pg.font.Font): 表示用フォント
            wallet (Wallet): 所持金オブジェクト
            cooldowns (List[int]): ユニットごとの残りクールダウン
            buttons (List[pg.Rect]): 出撃ボタンのRectリスト
            wallet_btn (pg.Rect): 財布レベルアップボタンのRect
        """
        # ユニット出撃ボタン描画
        for i, (spec, btn) in enumerate(zip(CAT_SPECS, buttons)):
            usable = cooldowns[i] == 0 and wallet.money >= spec["cost"]
            pg.draw.rect(screen, (255, 230, 120) if usable else (150, 150, 150), btn)
            if cooldowns[i] > 0:  # クールダウン中は残り時間を暗いゲージで示す
                h = int(btn.height * cooldowns[i] / spec["cooldown"])
                pg.draw.rect(screen, (90, 90, 90), (btn.left, btn.top, btn.width, h))
            pg.draw.rect(screen, (0, 0, 0), btn, 3)
            name = font.render(f"{i + 1}:{spec['name']}", True, (0, 0, 0))
            cost = font.render(f"{spec['cost']}円", True, (0, 0, 0))
            screen.blit(name, (btn.left + 10, btn.top + 10))
            screen.blit(cost, (btn.left + 10, btn.top + 50))

        # 財布レベルアップボタン描画
        cost_level_up = wallet.get_level_up_cost()
        is_max = wallet.level >= wallet.max_level
        can_level_up = not is_max and wallet.money >= cost_level_up

        pg.draw.rect(
            screen, (100, 220, 100) if can_level_up else (160, 160, 160), wallet_btn
        )
        pg.draw.rect(screen, (0, 0, 0), wallet_btn, 3)

        btn_lbl = font.render("[W/0]財布", True, (0, 0, 0))
        if is_max:
            cost_lbl = font.render("レベル MAX", True, (0, 0, 0))
        else:
            cost_lbl = font.render(f"{cost_level_up}円", True, (0, 0, 0))

        screen.blit(btn_lbl, (wallet_btn.left + 8, wallet_btn.top + 10))
        screen.blit(cost_lbl, (wallet_btn.left + 10, wallet_btn.top + 50))

        # 所持金および財布LV表示
        money_str = f"お金 {int(wallet.money)} / {wallet.max_money}円 (Lv.{wallet.level})"
        money = font.render(money_str, True, (0, 0, 0))
        screen.blit(money, (WIDTH - money.get_width() - 20, 20))


    def draw_field(screen: pg.Surface) -> None:
        """背景（空と地面）を描画する.

        Args:
            screen (pg.Surface): 画面Surface
        """
        screen.fill((135, 206, 235))
```

```
        pg.draw.rect(screen, (150, 110, 70), (0, GROUND_Y, WIDTH, HEIGHT - GROUND_Y))
        pg.draw.line(screen, (0, 0, 0), (0, GROUND_Y), (WIDTH, GROUND_Y), 3)



def show_result(screen: pg.Surface, font: pg.font.Font, message: str) -> None:
    """勝敗の結果を3秒間表示する.

    Args:
        screen (pg.Surface): 画面Surface
        font (pg.font.Font): 表示用フォント
        message (str): 表示する文字列
    """
    overlay = pg.Surface((WIDTH, HEIGHT))
    overlay.set_alpha(150)
    screen.blit(overlay, (0, 0))
    text = font.render(message, True, (255, 255, 255))
    screen.blit(text, text.get_rect(center=(WIDTH // 2, HEIGHT // 2)))
    pg.display.update()
    time.sleep(3)



def main() -> None:
    """メインループ関数."""
    pg.display.set_caption("ネコ大戦争（仮）")
    screen = pg.display.set_mode((WIDTH, HEIGHT))
    font = get_font(24)
    big_font = get_font(90)
    clock = pg.time.Clock()

    cat_base, enemy_base = Base(True), Base(False)
    cats, enemies = pg.sprite.Group(), pg.sprite.Group()
    wallet = Wallet()
    cooldowns = [0] * len(CAT_SPECS)
    buttons = make_buttons()
    wallet_btn = make_wallet_button()
    tmr = 0

    while True:
        for event in pg.event.get():
            if event.type == pg.QUIT:
                return

            if event.type == pg.KEYDOWN:
                # 数字キー 1〜3 で出撃
                if pg.K_1 <= event.key < pg.K_1 + len(CAT_SPECS):
                    try_spawn(event.key - pg.K_1, wallet, cooldowns, cats, cat_base)
                # Wキーまたは 0 キーで財布レベルアップ
                elif event.key in (pg.K_w, pg.K_0, pg.K_KP0):
                    wallet.level_up()

            if event.type == pg.MOUSEBUTTONDOWN and event.button == 1:
                # 出撃ボタン判定
                for i, btn in enumerate(buttons):
                    if btn.collidepoint(event.pos):
                        try_spawn(i, wallet, cooldowns, cats, cat_base)
                # 財布レベルアップボタン判定
                if wallet_btn.collidepoint(event.pos):
                    wallet.level_up()
```

```
            wallet.update()
            for i in range(len(cooldowns)):
                cooldowns[i] = max(0, cooldowns[i] - 1)
            if tmr % ENEMY_INTERVAL == ENEMY_INTERVAL - 1:
                spec = random.choices(ENEMY_SPECS, weights=[3, 1])[0]
                enemies.add(Unit(spec, False, enemy_base.rect.right))
            cats.update(enemies, enemy_base)
            enemies.update(cats, cat_base)

            draw_field(screen)
            enemy_base.draw(screen)
            cat_base.draw(screen)
            cats.draw(screen)
            enemies.draw(screen)
            for unit in list(cats) + list(enemies):
                if isinstance(unit, Unit):
                    draw_hp_bar(screen, unit.rect, unit.hp, unit.spec["hp"])
            draw_ui(screen, font, wallet, cooldowns, buttons, wallet_btn)

            if enemy_base.hp <= 0:
                show_result(screen, big_font, "勝利！")
                return
            if cat_base.hp <= 0:
                show_result(screen, big_font, "敗北…")
                return
            pg.display.update()
            tmr += 1
            clock.tick(FPS)


    if __name__ == "__main__":
        pg.init()
        main()
        pg.quit()
        sys.exit()
```

C0C25077 / ProjExD_Group16

Code Pull requests Actions Projects Wiki Security and quality More

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also compare across forks or learn more about diff comparisons.

base: main ← compare: wallet ✓ Able to merge. These branches can be automatically merged.

Discuss and review the changes in this comparison with others. Learn about pull requests **Create pull request**

2 commits 2 files changed 1 contributor

Commits on Nov 3, 2026

追加機能１（財布システム）実装完了
C0C25077 committed 20 minutes ago e8072bb

README追記
C0C25077 committed 6 minutes ago a33669c

Showing 2 changed files with **272 additions** and **108 deletions**. Split Unified

11 README.md

```
6   6   * pygame >= 2.1
7   7
8   8   ## ゲームの概要
9       - * （どんなゲームか）
    9   + * 右側の自陣からキャラクターを出撃させ、左側の敵陣を攻略するタワーディフェンスゲーム
10  10  * 参考URL：[サイトタイトル](https://...)
11  11
12  12  ## ゲームの遊び方
20  20  * 機能名（担当：名前）：説明
21  21  ・追加機能１（もろ）：お金上昇＆財布レベルアップ
22  22  ➡キーかボタンで所持金の上限と増加速度を強化。強化費用が上がり、最大レベルを設定する。UIにレベル表示
    23  + ・追加機能１（もろ）：お金上昇＆財布レベルアップ
    24  + ➡ キー入力やUIボタンクリックにより、お金の管理パラメータをリアルタイムで強化するシステム。
    25  +   **操作**:
    26  +     `W` キー / `0` キー / 画面右下「財布LV UP」ボタン
    27  +   **仕様**:
    28  +     `Wallet` クラスを拡張し、財布レベル（`level`）、最大レベル（`max_level`）、強化費用計算（`get_level_up_cost()`）、レベルアップ処理（`level_up()`）を実装。
    29  +     レベルアップごとに「所持金上限（`+350`）」と「毎フレームの収入量（`+0.35`）」が増加。
    30  +     レベルに応じたコスト（レベル×100円）を設定し、所持金が足りない場合や最大レベル到達時はボタンがグレーアウトして発動を制限。
    31  +     画面右上のお金表示に現在のレベルを追記し、ボタン内に操作キーと必要コストを描画。
```

```
23  32
24  33  ・追加機能２（かわひら）：キャラクター全般
25  34  ➡範囲攻撃、遠距離攻撃、味方の足を止める敵、体力が大きいボス敵など。Unit を継承したクラスで作る
```

369 nyanko base.py

```
...  ...  @@ -1,56 +1,105 @@
1    1    """
2         - にゃんこ大戦争風ゲームの共通基本機能（ベース部分）
     2    + にゃんこ大戦争風ゲーム
3    3
4    4    右側の自陣から味方ユニット（ネコ）を出撃させ，左側の敵陣を攻略するゲーム．
5    5    お金は時間とともに増え，お金とクールダウンの許す範囲でユニットを出撃できる．
6    6    ユニットは敵に当たると立ち止まって攻撃し，敵陣のHPを0にすれば勝利．
7    7
8         - 操作：数字キー1～3，または画面下のボタンのクリックでユニットを出撃
     8    + 操作：
     9    + - 数字キー 1～3 または画面下ボタン：ユニット出撃
     10   + - Wキー / 0キー または画面右下ボタン：財布レベルアップ（お金の増加速度と上限アップ）
9    11   """
     12   +
10   13   import os
11   14   import random
12   15   import sys
13   16   import time
     17   + from typing import List, Optional
14   18
15   19   import pygame as pg
16   20
17        -
     21   + # 定数定義
18   22   WIDTH, HEIGHT = 1100, 650
19   23   GROUND_Y = 500  # 地面の高さ（ユニットの足元のy座標）
20   24   FPS = 50
21   25   ENEMY_INTERVAL = 150  # 敵が出現する間隔（フレーム数）
22   26   BASE_HP = 1000
     27   +
23   28   os.chdir(os.path.dirname(os.path.abspath(__file__)))
24   29
25   30   # 味方ユニットの性能（追加機能として種類を増やしやすいように辞書で管理）
26   31   CAT_SPECS = [
27        -     {"name": "ネコ", "cost": 50, "hp": 100, "atk": 10, "speed": 2.0,
28        -      "range": 20, "interval": 40, "cooldown": 100,
29        -      "color": (255, 220, 160), "size": (40, 50)},
30        -     {"name": "タンク", "cost": 100, "hp": 500, "atk": 6, "speed": 1.2,
31        -      "range": 20, "interval": 50, "cooldown": 200,
32        -      "color": (200, 200, 255), "size": (50, 70)},
33        -     {"name": "キック", "cost": 150, "hp": 120, "atk": 35, "speed": 1.8,
34        -      "range": 60, "interval": 70, "cooldown": 250,
35        -      "color": (255, 150, 150), "size": (40, 60)},
     32   +     {
     33   +         "name": "ネコ",
     34   +         "cost": 50,
     35   +         "hp": 100,
     36   +         "atk": 10,
     37   +         "speed": 2.0,
     38   +         "range": 20,
```

```
+         "interval": 40,
+         "cooldown": 100,
+         "color": (255, 220, 160),
+         "size": (40, 50),
+     },
+     {
+         "name": "タンク",
+         "cost": 100,
+         "hp": 500,
+         "atk": 6,
+         "speed": 1.2,
+         "range": 20,
+         "interval": 50,
+         "cooldown": 200,
+         "color": (200, 200, 255),
+         "size": (50, 70),
+     },
+     {
+         "name": "キック",
+         "cost": 150,
+         "hp": 120,
+         "atk": 35,
+         "speed": 1.8,
+         "range": 60,
+         "interval": 70,
+         "cooldown": 250,
+         "color": (255, 150, 150),
+         "size": (40, 60),
+     },
  ]

  # 敵ユニットの性能
  ENEMY_SPECS = [
-     {"name": "ワンコ", "hp": 120, "atk": 8, "speed": 1.5,
-      "range": 20, "interval": 45,
-      "color": (240, 240, 240), "size": (40, 40)},
-     {"name": "ブタ", "hp": 500, "atk": 20, "speed": 1.0,
-      "range": 20, "interval": 60,
-      "color": (255, 170, 200), "size": (60, 80)},
+     {
+         "name": "ワンコ",
+         "hp": 120,
+         "atk": 8,
+         "speed": 1.5,
+         "range": 20,
+         "interval": 45,
+         "color": (240, 240, 240),
+         "size": (40, 40),
+     },
+     {
+         "name": "ブタ",
+         "hp": 500,
+         "atk": 20,
+         "speed": 1.0,
+         "range": 20,
+         "interval": 60,
+         "color": (255, 170, 200),
+         "size": (60, 80),
```

```
                 +     },
46               ]
47
48
49               def get_font(size: int) -> pg.font.Font:
50               -     """
51               -     日本語を表示できるフォントを探して返す
52               -     引数 size：フォントサイズ
53               -     戻り値：Fontオブジェクト（見つからない場合は標準フォント）
                 +     """日本語を表示できるフォントを探して返す.
                 +
                 +     Args:
                 +         size (int): フォントサイズ
                 +
                 +     Returns:
                 +         pg.font.Font: Fontオブジェクト（見つからない場合は標準フォント）
54                       """
55                       name = pg.font.match_font(
56                           "meiryo,yugothic,msgothic,notosansscjkjp,hiraginosans,takaopgothic"
59
60
61               def draw_hp_bar(screen: pg.Surface, rct: pg.Rect, hp: int, max_hp: int) -> None:
62               -     """
63               -     対象の頭上にHPバーを描画する
64               -     引数 screen：画面Surface
65               -     引数 rct：対象のRect
66               -     引数 hp：現在のHP
67               -     引数 max_hp：最大HP
                 +     """対象の頭上にHPバーを描画する.
                 +
                 +     Args:
                 +         screen (pg.Surface): 画面Surface
                 +         rct (pg.Rect): 対象のRect
                 +         hp (int): 現在のHP
                 +         max_hp (int): 最大HP
68                       """
69                       ratio = max(0, hp) / max_hp
70                       bar = pg.Rect(rct.left, rct.top - 10, rct.width, 6)
71                       pg.draw.rect(screen, (80, 80, 80), bar)
72               -     pg.draw.rect(screen, (0, 200, 0), (bar.left, bar.top, int(bar.width * ratio),
                 bar.height))
                 +     pg.draw.rect(
                 +         screen,
                 +         (0, 200, 0),
                 +         (bar.left, bar.top, int(bar.width * ratio), bar.height),
                 +     )
73
74
75               class Base:
76               -     """
77               -     自陣・敵陣（城）に関するクラス
78               -     """
                 +     """自陣・敵陣（城）に関するクラス."""
                 +
79                       def __init__(self, is_cat: bool) -> None:
80               -         """
81               -         引数 is_cat：Trueなら自陣（右側），Falseなら敵陣（左側）
82               -         """
```

```
                133 +          """引数 is_cat：Trueなら自陣（右側），Falseなら敵陣（左側）."""
83              134            self.hp = BASE_HP
84              135            self.max_hp = BASE_HP
85              136            self.image = pg.Surface((80, 200))
90              141            self.rect.centerx = WIDTH - 80 if is_cat else 80
91              142
92              143        def draw(self, screen: pg.Surface) -> None:
93                  -          """
94                  -          城とHPバーを描画する
95                  -          引数 screen：画面Surface
                144 +          """城とHPバーを描画する.
                145 +
                146 +          Args:
                147 +              screen (pg.Surface): 画面Surface
96              148            """
97              149            screen.blit(self.image, self.rect)
98              150            draw_hp_bar(screen, self.rect, self.hp, self.max_hp)
99              151
100             152
101             153    class Unit(pg.sprite.Sprite):
102                 -      """
103                 -      味方・敵のユニットに関するクラス
104                 -      """
                154 +      """味方・敵のユニットに関するクラス."""
                155 +
105             156        def __init__(self, spec: dict, is_cat: bool, x: int) -> None:
106                 -          """
107                 -          引数 spec：ユニットの性能を表す辞書
108                 -          引数 is_cat：Trueなら味方（左へ進む），Falseなら敵（右へ進む）
109                 -          引数 x：出現位置のx座標（味方は右端，敵は左端をこの座標に合わせる）
                157 +          """ユニット初期化.
                158 +
                159 +          Args:
                160 +              spec (dict): ユニットの性能を表す辞書
                161 +              is_cat (bool): Trueなら味方（左へ進む），Falseなら敵（右へ進む）
                162 +              x (int): 出現位置のx座標
110             163            """
111             164            super().__init__()
112             165            self.spec = spec
125             178            self.x = float(self.rect.centerx)  # 小数の速度を扱うため，x座標は別に保持する
126             179
127             180        def can_reach(self, rct: pg.Rect) -> bool:
128                 -          """
129                 -          対象のRectが攻撃射程内にあるかを判定する
130                 -          引数 rct：敵ユニットまたは敵陣のRect
131                 -          戻り値：射程内ならTrue
                181 +          """対象のRectが攻撃射程内にあるかを判定する.
                182 +
                183 +          Args:
                184 +              rct (pg.Rect): 敵ユニットまたは敵陣のRect
                185 +
                186 +          Returns:
                187 +              bool: 射程内ならTrue
132             188            """
133             189            if self.dir < 0:
134             190                return rct.right >= self.rect.left - self.spec["range"]
135             191            return rct.left <= self.rect.right + self.spec["range"]
136             192
```

```
137      -     def find_target(self, opponents: pg.sprite.Group, base: Base) -> "Unit | Base |
               None":
138      -         """
139      -         攻撃対象を探す（射程内のユニットのうち最も近いもの，いなければ敵城）
140      -         引数 opponents：相手側ユニットのグループ
141      -         引数 base：相手側の城
142      -         戻り値：攻撃対象（いなければNone）
         193  +     def find_target(
         194  +         self, opponents: pg.sprite.Group, base: Base
         195  +     ) -> Optional["Unit | Base"]:
         196  +         """攻撃対象を探す（射程内のユニットのうち最も近いもの，いなければ敵城）.
         197  +
         198  +         Args:
         199  +             opponents (pg.sprite.Group): 相手側ユニットのグループ
         200  +             base (Base): 相手側の城
         201  +
         202  +         Returns:
         203  +             Optional[Unit | Base]: 攻撃対象（いなければNone）
143      204              """
144      205              cands = [o for o in opponents if self.can_reach(o.rect)]
145      206              if cands:
149      210              return None
150      211
151      212          def update(self, opponents: pg.sprite.Group, base: Base) -> None:
152      -         """
153      -         HP判定，攻撃，移動を行う
154      -         引数 opponents：相手側ユニットのグループ
155      -         引数 base：相手側の城
         213  +         """HP判定，攻撃，移動を行う.
         214  +
         215  +         Args:
         216  +             opponents (pg.sprite.Group): 相手側ユニットのグループ
         217  +             base (Base): 相手側の城
156      218              """
157      219              if self.hp <= 0:
158      220                  self.kill()
169      231
170      232
171      233  class Wallet:
172      -     """
173      -     お金（所持金）に関するクラス
174      -     """
         234  +     """お金（所持金）および財布のレベルアップに関するクラス."""
         235  +
175      236          def __init__(self) -> None:
         237  +         """初期パラメータの設定."""
176      238              self.money = 100.0
177      -         self.max_money = 1000
178      -         self.income = 0.5  # 1フレームあたりの増加量
         239  +         self.level = 1
         240  +         self.max_level = 8
         241  +         self.max_money = 500  # 初期上限
         242  +         self.income = 0.4  # 1フレームあたりの初期増加量
179      243
180      244          def update(self) -> None:
181      -         """
182      -         お金を時間経過で増やす（上限あり）
183      -         """
```

```
184        -         self.money = min(self.max_money, self.money + self.income)
           245 +     """お金を時間経過で増やす（上限あり）."""
           246 +     self.money = min(float(self.max_money), self.money + self.income)
185        247
186        248     def pay(self, cost: int) -> bool:
187        -         """
188        -         所持金が足りれば支払う
189        -         引数 cost：支払う金額
190        -         戻り値：支払えたらTrue，足りなければFalse
           249 +     """所持金が足りれば支払う.
           250 +
           251 +     Args:
           252 +         cost (int): 支払う金額
           253 +
           254 +     Returns:
           255 +         bool: 支払えたらTrue，足りなければFalse
191        256         """
192        257         if self.money < cost:
193        258             return False
194        259         self.money -= cost
195        260         return True
196        261
           262 +     def get_level_up_cost(self) -> int:
           263 +         """現在のレベルに応じたレベルアップに必要な費用を取得する.
197        264
198        - def make_buttons() -> list[pg.Rect]:
199        -     """
200        -     出撃ボタンのRectのリストを作る
201        -     戻り値：味方ユニットの種類ごとのボタンRect
           265 +         Returns:
           266 +             int: レベルアップに必要な費用（最大レベルの場合は0）
           267 +         """
           268 +         if self.level >= self.max_level:
           269 +             return 0
           270 +         return 100 * self.level
           271 +
           272 +     def level_up(self) -> bool:
           273 +         """所持金が足りれば財布レベルを上げ，所持金上限と増加速度を強化する.
           274 +
           275 +         Returns:
           276 +             bool: レベルアップに成功したらTrue，失敗・レベルMAXならFalse
           277 +         """
           278 +         if self.level >= self.max_level:
           279 +             return False
           280 +         cost = self.get_level_up_cost()
           281 +         if not self.pay(cost):
           282 +             return False
           283 +
           284 +         self.level += 1
           285 +         self.max_money += 350  # 所持金上限の増加
           286 +         self.income += 0.35   # 毎フレームの収入増加量アップ
           287 +         return True
           288 +
           289 +
           290 + def make_buttons() -> List[pg.Rect]:
           291 +     """出撃ボタンのRectのリストを作る.
           292 +
           293 +     Returns:
```

```
                294 +         List[pg.Rect]: 味方ユニットの種類ごとのボタンRect
202             295       """
203             296       return [pg.Rect(20 + i * 170, 540, 160, 90) for i in range(len(CAT_SPECS))]
204             297
205             298
206               - def try_spawn(idx: int, wallet: Wallet, cooldowns: list[int],
207               -               cats: pg.sprite.Group, cat_base: Base) -> None:
                299 + def make_wallet_button() -> pg.Rect:
                300 +     """財布レベルアップボタンのRectを作る.
                301 +
                302 +     Returns:
                303 +         pg.Rect: 財布レベルアップボタンのRect
208             304       """
209               -     お金とクールダウンの条件を満たせば，味方ユニットを出撃させる
210               -     引数 idx：CAT_SPECSの番号
211               -     引数 wallet：所持金
212               -     引数 cooldowns：ユニットごとの残りクールダウン（フレーム数）
213               -     引数 cats：味方ユニットのグループ
214               -     引数 cat_base：自陣
                305 +     return pg.Rect(WIDTH - 180, 540, 160, 90)
                306 +
                307 +
                308 + def try_spawn(
                309 +     idx: int,
                310 +     wallet: Wallet,
                311 +     cooldowns: List[int],
                312 +     cats: pg.sprite.Group,
                313 +     cat_base: Base,
                314 + ) -> None:
                315 +     """お金とクールダウンの条件を満たせば，味方ユニットを出撃させる.
                316 +
                317 +     Args:
                318 +         idx (int): CAT_SPECSの番号
                319 +         wallet (Wallet): 所持金オブジェクト
                320 +         cooldowns (List[int]): ユニットごとの残りクールダウン
                321 +         cats (pg.sprite.Group): 味方ユニットのグループ
                322 +         cat_base (Base): 自陣
215             323       """
216             324       spec = CAT_SPECS[idx]
217             325       if cooldowns[idx] > 0 or not wallet.pay(spec["cost"]):
220             328       cooldowns[idx] = spec["cooldown"]
221             329
222             330
223               - def draw_ui(screen: pg.Surface, font: pg.font.Font, wallet: Wallet,
224               -             cooldowns: list[int], buttons: list[pg.Rect]) -> None:
225               -     """
226               -     所持金表示と出撃ボタンを描画する
227               -     引数 screen：画面Surface
228               -     引数 font：表示用フォント
229               -     引数 wallet：所持金
230               -     引数 cooldowns：ユニットごとの残りクールダウン
231               -     引数 buttons：出撃ボタンのRectリスト
                331 + def draw_ui(
                332 +     screen: pg.Surface,
                333 +     font: pg.font.Font,
                334 +     wallet: Wallet,
                335 +     cooldowns: List[int],
                336 +     buttons: List[pg.Rect],
```

```
                337 +     wallet_btn: pg.Rect,
                338 + ) -> None:
                339 +     """所持金表示・出撃ボタン・財布レベルアップボタンを描画する.
                340 +
                341 +     Args:
                342 +         screen (pg.Surface): 画面Surface
                343 +         font (pg.font.Font): 表示用フォント
                344 +         wallet (Wallet): 所持金オブジェクト
                345 +         cooldowns (List[int]): ユニットごとの残りクールダウン
                346 +         buttons (List[pg.Rect]): 出撃ボタンのRectリスト
                347 +         wallet_btn (pg.Rect): 財布レベルアップボタンのRect
232             348       """
                349 +     # ユニット出撃ボタン描画
233             350       for i, (spec, btn) in enumerate(zip(CAT_SPECS, buttons)):
234             351           usable = cooldowns[i] == 0 and wallet.money >= spec["cost"]
235             352           pg.draw.rect(screen, (255, 230, 120) if usable else (150, 150, 150), btn)
241             358           cost = font.render(f"{spec['cost']}円", True, (0, 0, 0))
242             359           screen.blit(name, (btn.left + 10, btn.top + 10))
243             360           screen.blit(cost, (btn.left + 10, btn.top + 50))
244             -     money = font.render(f"お金 {int(wallet.money)} / {wallet.max_money}円", True, (0,
                    0, 0))
                361 +
                362 +     # 財布レベルアップボタン描画
                363 +     cost_level_up = wallet.get_level_up_cost()
                364 +     is_max = wallet.level >= wallet.max_level
                365 +     can_level_up = not is_max and wallet.money >= cost_level_up
                366 +
                367 +     pg.draw.rect(
                368 +         screen, (100, 220, 100) if can_level_up else (160, 160, 160), wallet_btn
                369 +     )
                370 +     pg.draw.rect(screen, (0, 0, 0), wallet_btn, 3)
                371 +
                372 +     btn_lbl = font.render("[W/0]財布", True, (0, 0, 0))
                373 +     if is_max:
                374 +         cost_lbl = font.render("レベル MAX", True, (0, 0, 0))
                375 +     else:
                376 +         cost_lbl = font.render(f"{cost_level_up}円", True, (0, 0, 0))
                377 +
                378 +     screen.blit(btn_lbl, (wallet_btn.left + 8, wallet_btn.top + 10))
                379 +     screen.blit(cost_lbl, (wallet_btn.left + 10, wallet_btn.top + 50))
                380 +
                381 +     # 所持金および財布LV表示
                382 +     money_str = f"お金 {int(wallet.money)} / {wallet.max_money}円 (Lv.{wallet.level})"
                383 +     money = font.render(money_str, True, (0, 0, 0))
245             384       screen.blit(money, (WIDTH - money.get_width() - 20, 20))
246             385
247             386
248             387   def draw_field(screen: pg.Surface) -> None:
249             -     """
250             -     背景（空と地面）を描画する
251             -     引数 screen：画面Surface
                388 +     """背景（空と地面）を描画する.
                389 +
                390 +     Args:
                391 +         screen (pg.Surface): 画面Surface
252             392       """
253             393       screen.fill((135, 206, 235))
254             394       pg.draw.rect(screen, (150, 110, 70), (0, GROUND_Y, WIDTH, HEIGHT - GROUND_Y))
```

```
255  395        pg.draw.line(screen, (0, 0, 0), (0, GROUND_Y), (WIDTH, GROUND_Y), 3)
256  396
257  397
258  398    def show_result(screen: pg.Surface, font: pg.font.Font, message: str) -> None:
259       -    """
260       -    勝敗の結果を3秒間表示する
261       -    引数 screen：画面Surface
262       -    引数 font：表示用フォント
263       -    引数 message：表示する文字列
     399  +    """勝敗の結果を3秒間表示する.
     400  +
     401  +    Args:
     402  +        screen (pg.Surface): 画面Surface
     403  +        font (pg.font.Font): 表示用フォント
     404  +        message (str): 表示する文字列
264  405        """
265  406        overlay = pg.Surface((WIDTH, HEIGHT))
266  407        overlay.set_alpha(150)
272  413
273  414
274  415    def main() -> None:
     416  +    """メインループ関数."""
275  417        pg.display.set_caption("ネコ大戦争（仮）")
276  418        screen = pg.display.set_mode((WIDTH, HEIGHT))
277       -    font = get_font(26)
     419  +    font = get_font(24)
278  420        big_font = get_font(90)
279  421        clock = pg.time.Clock()
280  422
283  425        wallet = Wallet()
284  426        cooldowns = [0] * len(CAT_SPECS)
285  427        buttons = make_buttons()
     428  +    wallet_btn = make_wallet_button()
286  429        tmr = 0
287  430
288  431        while True:
289  432            for event in pg.event.get():
290  433                if event.type == pg.QUIT:
291  434                    return
292       -            if event.type == pg.KEYDOWN and pg.K_1 <= event.key < pg.K_1 +
     len(CAT_SPECS):
293       -                try_spawn(event.key - pg.K_1, wallet, cooldowns, cats, cat_base)
     435  +
     436  +            if event.type == pg.KEYDOWN:
     437  +                # 数字キー 1〜3 で出撃
     438  +                if pg.K_1 <= event.key < pg.K_1 + len(CAT_SPECS):
     439  +                    try_spawn(event.key - pg.K_1, wallet, cooldowns, cats, cat_base)
     440  +                # Wキーまたは 0 キーで財布レベルアップ
     441  +                elif event.key in (pg.K_w, pg.K_0, pg.K_KP0):
     442  +                    wallet.level_up()
     443  +
294  444                if event.type == pg.MOUSEBUTTONDOWN and event.button == 1:
     445  +                # 出撃ボタン判定
295  446                    for i, btn in enumerate(buttons):
296  447                        if btn.collidepoint(event.pos):
297  448                            try_spawn(i, wallet, cooldowns, cats, cat_base)
     449  +                # 財布レベルアップボタン判定
     450  +                if wallet_btn.collidepoint(event.pos):
```

```
      451 +                    wallet.level_up()
298   452
299   453          wallet.update()
300   454          for i in range(len(cooldowns)):
311   465          cats.draw(screen)
312   466          enemies.draw(screen)
313   467          for unit in list(cats) + list(enemies):
314       -            draw_hp_bar(screen, unit.rect, unit.hp, unit.spec["hp"])
315       -        draw_ui(screen, font, wallet, cooldowns, buttons)
      468 +            if isinstance(unit, Unit):
      469 +                draw_hp_bar(screen, unit.rect, unit.hp, unit.spec["hp"])
      470 +        draw_ui(screen, font, wallet, cooldowns, buttons, wallet_btn)
316   471
317   472          if enemy_base.hp <= 0:
318   473              show_result(screen, big_font, "勝利！")
```